THE MILLS KORNER

Master Full-Stack Website Project Handoff

React + Django REST Framework + PostgreSQL

Purpose of this document

This is the canonical handoff context for continuing the TMK project in a new ChatGPT conversation. It is intentionally detailed rather than condensed. A new assistant should be able to understand the client vision, engineering decisions, security philosophy, planned data model, backend and frontend architecture, phased delivery plan, unresolved questions, and exact next step without relying on the prior chat.

Status: Planning and architecture are complete enough to begin implementation. No production code has been written yet in this handoff.

Current resume/portfolio objective: build a real first-client project that is excellent for TMK and also demonstrates professional Python full-stack engineering, backend quality, security-conscious design, database design, API architecture, testing, Git workflow, and deployment.

How to Use This Handoff

Upload this document into a new ChatGPT conversation and say: "Continue the TMK project from this handoff. Start at the exact next step and preserve the architecture/security decisions unless a new client requirement justifies changing them."

The assistant should not treat every idea below as final client-approved copy. The document explicitly distinguishes client direction, engineering decisions, and unresolved questions.

Canonical continuation rules

- Work step-by-step, but detailed explanations are welcome; the developer is comfortable with depth when it is clear and simple.
- Do not skip setup steps or assume files already exist.
- For coding phases, provide exact commands, paths, files, and code, then explain what each major block does and why it exists.
- Favor professional, realistic architecture over tutorial shortcuts.
- Security is a first-class requirement throughout the project.
- Do not add technologies only to make the resume look impressive; every technology must solve a real requirement.
- Do not overbuild the MVP. Preserve future expansion paths without delaying delivery.
- Use Git/GitHub professionally and recommend commit points/messages throughout implementation.
- Use Postman and automated tests to verify APIs and security boundaries.
- At project completion, produce resume bullets and interview explanations of architecture, security, database design, and tradeoffs.

1. Client Vision and Product Intent

The Mills Korner (TMK) client guideline does not read like a traditional technical specification. It is primarily a strategic brand, content, and user-experience vision. The engineering team must translate that vision into software requirements without inventing final client data.

Core organizing idea

> **Opportunity changes everything.**
> The site should communicate one memorable institutional idea rather than making visitors remember a dozen disconnected programs.

The client describes the desired site as feeling more like a headquarters than a generic website: coherent, intentional, premium, and reinforcing one idea across every page, image, statistic, story, and call to action.

Desired visitor emotional journey

```
I understand the problem.
        ↓
I understand why TMK exists.
        ↓
I believe TMK is solving it.
        ↓
I trust TMK.
        ↓
I want to participate.
```

Participation should feel earned through trust and belief, not produced by constant calls to donate or volunteer.

Public audience

- Young people and families
- Parents
- Prospective mentors and volunteers
- Corporate partners and CEOs
- Foundation leaders
- Schools and community partners
- Potential donors/investors/supporters
- General community visitors

Institutional impression

The lasting impression should be that The Mills Korner consistently creates opportunities for young people to become who they are capable of becoming. The site should present TMK as an institution with measurable impact, credible partnerships, authentic stories, and a clear pathway to participate.

2. Client Design and Content Direction

Public visual personality

- Elegant, quiet, confident, premium, institutional.
- Large authentic photography.
- Large typography.
- Substantial whitespace.
- Minimal scrolling clutter and restrained copy.
- Real moments rather than posed stock imagery.
- One elegant serif typeface plus one clean sans-serif typeface.
- Forest Green, Warm Ivory, Brushed Gold, and Charcoal as the working palette direction.
- Warm color and black-and-white photography can be mixed intentionally.
- Avoid clip art, cartoons, script fonts, random bright colors, generic SaaS gradients, and noisy card-heavy nonprofit templates.

Photography direction

- No generic stock photography.
- Prefer a mentor beside a student, a coach kneeling beside an athlete, a financial-literacy workshop, a volunteer embracing a family, and similar authentic moments.
- Use high-resolution client-approved images with appropriate consent/usage rights, especially because TMK serves young people.
- Optimize images for performance and accessibility rather than serving original multi-megabyte uploads directly.

Hero concept from client guideline

```
Developing Leaders. Expanding Opportunity.
Strengthening Communities.

The Mills Korner equips young people ages 10–22 with
the relationships, skills, and opportunities they need
to build successful futures.

Become a Mentor | Partner With Us | Invest in a Future
```

Important: the client guideline contains alternate wording elsewhere. Treat the above as concept direction until final copy is approved.

Homepage flow

1. Hero
2. Impact immediately after hero
3. Opportunity areas
4. Core philosophy
5. Transformation stories
6. Partners
7. Invest / Get Involved call to action

8. Footer

Impact-first philosophy

The client explicitly prefers impact before a program catalog. The homepage should establish evidence and credibility quickly.

Opportunity framing

The client direction favors organizing the work around opportunity pathways rather than making visitors memorize programs. Working opportunity areas include Academic Success, Financial Capability, Leadership Development, Wellness/Resilience, Mentorship, and Athletics as Leadership. Athletics should be treated as a pathway/tool within leadership development, not the organization's entire identity.

Transformation stories

Use stories rather than generic testimonials. The desired structure is real person, real photography/video, before, transformation, and where they are now. Quality is preferred over quantity; the guideline suggests a small number of strong stories.

Partners and investment

Partners may include schools, foundations, businesses, cities, and faith communities. The support language can emphasize investing in opportunity rather than relying only on generic donation language.

3. Product Definition

This is not being built as a static brochure site. The target is a secure, content-driven full-stack platform with a polished public experience and real administrative/business functionality.

Surface	Primary users	Purpose
Public React Website	Visitors, families, partners, donors, volunteers	Communicate TMK identity, impact, opportunity pathways, stories, events, and ways to participate.
React Staff Portal	Authorized TMK staff	Manage content and business data through protected REST APIs. Planned custom client-facing dashboard.
Django Admin	Internal authorized staff	Reliable fallback/internal administration using Django session authentication and built-in permissions.

High-level request flow

```
PUBLIC
Visitor -> React -> Public DRF API -> PostgreSQL

STAFF PORTAL
Authorized staff -> React -> JWT -> Protected DRF API -> PostgreSQL

INTERNAL FALLBACK
Authorized staff -> Django /admin/ -> Session Authentication -> PostgreSQL
```

Critical security boundary

React may hide/show buttons and protect routes for user experience, but React is never the final authorization boundary. Django/DRF must authenticate and authorize every protected request independently.

4. Technology Stack and Why

Layer	Decision	Reason
Frontend	React + Vite	Strong resume value, reusable component architecture, interactive public site and staff SPA experience.
Backend	Python + Django + Django REST Framework	Excellent fit for content management, relational data, authentication, permissions, admin fallback, file handling, validation, and mature API development.
Database	PostgreSQL	Professional relational database, strong Django integration, constraints, indexing, JSON support when appropriate, managed hosting options.
API	REST under /api/v1/	Clear separation between React and backend, versionable contract, easy Postman testing.
Staff SPA auth	JWT access/refresh strategy	Appropriate for a separate React client consuming protected APIs.
Django Admin auth	Django sessions	Correct fit for Django's server-rendered admin; JWT adds no useful security benefit there.
Version control	Git + GitHub	Professional collaboration/history and portfolio visibility where client permissions allow.
API testing	Postman + automated tests	Manual verification plus repeatable regression/security tests.
Deployment	Vercel + Render/Railway or equivalent	Beginner-friendly managed deployment; final provider choice can change based on client needs/cost.

Why Django/DRF over Flask or FastAPI for this client

Flask and FastAPI are valid Python technologies, but Django/DRF is the chosen fit for TMK because this project naturally needs relational content, staff accounts, permissions, administration, forms, image/file handling, auditability, and mature security defaults. The portfolio can later include FastAPI in another project to broaden experience rather than forcing it into this client system.

Why not overengineer

Do not add Kubernetes, microservices, Kafka, GraphQL, Redis clusters, or elaborate cloud infrastructure without a real requirement. Professional engineering means choosing the simplest architecture that reliably solves the client's needs and can be maintained.

5. Public Website Information Architecture

Page	Purpose
Home	Explain what TMK is, establish impact/trust, and guide the visitor toward participation.
About	Story, mission, vision, leadership/founder context.
Impact	Public database-driven impact dashboard and evidence of outcomes.
How We Build Opportunity	Explain the opportunity pathways rather than a disconnected program list.
Stories	Transformation stories with authentic narrative and media.
Partners	Show trusted institutions and organizations working with TMK.
Events	Upcoming/past events and event detail pages.
Get Involved	Mentor, volunteer, partner, and support/invest pathways.
Contact	Secure general contact flow plus official contact/social information.

Proposed public React routes

```
/
 /about
 /impact
 /opportunity
 /opportunity/:slug
 /stories
 /stories/:slug
 /partners
 /events
 /events/:slug
 /get-involved
 /contact
```

Homepage component concept

```
HomePage
├── Navbar
├── HeroSection
├── ImpactPreview
├── OpportunityPreview
├── PhilosophySection
├── FeaturedStories
├── PartnerStrip
├── InvestCTA
└── Footer
```

6. Staff Portal Vision

The staff portal is intended to become real client-facing business software, not merely a portfolio add-on. It should be visually related to the TMK brand but more functional and information-dense than the public site.

Proposed staff routes

```
/staff/login
/staff/dashboard
/staff/impact
/staff/opportunities
/staff/stories
/staff/partners
/staff/events
/staff/inquiries
/staff/audit
/staff/profile
```

Dashboard information candidates

- New inquiries
- Upcoming events
- Published stories
- Partner count
- Recent content changes
- Recent audit activity
- Last website update

Delivery discipline

Django Admin comes first

Django Admin should be fully usable before the custom React staff portal is complete. This gives TMK a reliable management interface early and prevents the custom dashboard from delaying the public website. The React staff portal is Phase 2 unless implementation progress/client priorities justify bringing parts forward.

7. Database Design - Core MVP Models

The schema should represent real business concepts and remain intentionally compact. Do not add tables or relationships only to make the system look enterprise-like.

7.1 User

Purpose: authorized TMK staff accounts, authentication, roles/permissions, and audit attribution. Use a custom Django user model from the beginning, before the first migration. Email should be the login identifier.

```
User
id
email                   UNIQUE, required
password                 Django-managed password hash
first_name
last_name
role
is_active
is_staff
is_superuser
created_at
updated_at
last_login
```

Working role candidates: SUPER_ADMIN, CONTENT_MANAGER, STAFF, VIEWER. Final role semantics should be kept simple and adjusted to real client responsibilities.

7.2 OpportunityArea

```
OpportunityArea
id
title
slug                    UNIQUE, required
short_description
description
featured_image
display_order
is_published
created_at
updated_at
```

Working examples: Academic Success, Financial Capability, Leadership Development, Wellness & Resilience, Mentorship, Athletics as Leadership.

7.3 ImpactMetric

```
ImpactMetric
id
name
value                   numeric/structured
prefix
suffix
description
display_order
is_featured
is_published
created_at
updated_at
```

Store structured values instead of strings such as '15,487+'. Example: value=15487, suffix='+'. This supports validation, formatting, charts, sorting, and future historical reporting.

7.4 Story

```
Story
id
title
slug                    UNIQUE
person_name              privacy-sensitive; may be first name/initial/etc.
summary
before_story
transformation_story
current_story
featured_image
video_url
is_featured
is_published
published_at
created_by               FK -> User
updated_by               FK -> User
created_at
updated_at
```

Because stories may involve youth, public identity must follow client consent/privacy rules. Do not assume full names should be public.

7.5 Partner

```
Partner
id
name
partner_type
logo
website_url
description
display_order
is_featured
is_published
created_at
updated_at
```

Working partner types: SCHOOL, FOUNDATION, BUSINESS, CITY, FAITH_ORGANIZATION, OTHER.

7.6 Event

```
Event
id
title
slug
summary
description
location_name
address
start_datetime
end_datetime
featured_image
registration_url
is_featured
is_published
```

```
created_by              FK -> User
updated_by              FK -> User
created_at
updated_at
```

7.7 Inquiry

```
Inquiry
id
inquiry_type
first_name
last_name
email
phone
organization
message
status
assigned_to             FK -> User, nullable
created_at
updated_at
```

MVP inquiry types: CONTACT, MENTOR, VOLUNTEER, PARTNERSHIP. Working statuses: NEW, IN_REVIEW, CONTACTED, CLOSED. Use one table initially instead of four nearly identical form tables. Split a workflow into its own model later only if requirements become materially different.

7.8 NewsletterSubscriber

```
NewsletterSubscriber
id
email                   UNIQUE
is_active
subscribed_at
unsubscribed_at
```

This may later become thinner or be replaced by an external newsletter provider integration.

7.9 AuditLog

```
AuditLog
id
user                    FK -> User
action
resource_type
resource_id
resource_name
old_values
new_values
ip_address
created_at
```

Working actions: CREATE, UPDATE, DELETE, LOGIN, LOGIN_FAILED, LOGOUT, PUBLISH, UNPUBLISH. Audit logs should be append-oriented and not normally editable through staff CRUD screens.

8. Database Relationships and Integrity

```
User 1 ——— * Story        (created_by / updated_by)
User 1 ——— * Event        (created_by / updated_by)
User 1 ——— * Inquiry      (assigned_to)
User 1 ——— * AuditLog     (user)

OpportunityArea          independent in MVP
ImpactMetric             independent in MVP
Partner                  independent in MVP
NewsletterSubscriber     independent in MVP
```

Integrity principles

- User.email unique and required.
- OpportunityArea.slug unique and required.
- Story.slug unique.
- NewsletterSubscriber.email unique.
- Required business fields should have model/database constraints.
- Validate impact values against meaningful ranges.
- Do not casually hard-delete important business records.
- Audit records should not be mutable through normal staff screens.
- Add indexes when query patterns justify them; do not pre-optimize blindly.

Models intentionally postponed

Program: not required for MVP because the client currently frames the organization around opportunity pathways. Add Program with a foreign key to OpportunityArea only if TMK identifies distinct programs under those areas.

MediaAsset: postpone a reusable media library until media-management requirements are clearer. Direct image/logo fields are sufficient initially.

Soft deletion

Full generic soft-delete infrastructure is not required initially. However, important business records such as inquiries/audit history should not be casually destroyed. Publishing/archive flags may be preferable for content.

9. Django Backend Architecture

Django apps by business domain

```
backend/
├── accounts/       # User, auth, roles, permissions
├── content/        # OpportunityArea, Story, Partner
├── impact/         # ImpactMetric
├── events/         # Event
├── engagement/     # Inquiry, NewsletterSubscriber
├── audit/          # AuditLog + centralized audit helpers
└── config/         # project settings, root URLs, ASGI/WSGI
```

Do not create one Django app per model and do not put everything in one giant api app. Group related business concepts.

Typical app structure

```
app/
├── admin.py
├── apps.py
├── models.py
├── serializers.py
├── permissions.py     # where domain-specific permissions are warranted
├── views.py
├── urls.py
├── services.py        # only when real business logic warrants it
└── tests/
```

Service layer rule

Do not create services.py merely to look enterprise-like. Introduce a service when a business operation spans models, has workflow logic, needs transaction boundaries, or benefits from reuse. Simple CRUD can remain close to serializers/views/models.

Settings structure

```
config/
├── settings/
│   ├── __init__.py
│   ├── base.py
│   ├── development.py
│   └── production.py
├── urls.py
├── asgi.py
└── wsgi.py
```

Development can use DEBUG=True. Production must use DEBUG=False, production hosts/origins, secure cookie settings, HTTPS-related settings, and production database/media configuration.

10. Serializer, View, Permission, and API Strategy

Serializer strategy

Do not automatically expose every model field. Public and staff representations may differ.

```
PostgreSQL -> Django Model -> Serializer -> JSON -> React
```

Example: PublicStorySerializer should expose only public content. StaffStorySerializer may additionally expose publishing state and management metadata. Internal notes, permission fields, or sensitive data should never leak simply because they exist on the model.

ViewSets vs focused views

Use DRF ViewSets for normal resource CRUD. Use focused API/generic views for workflow actions such as login, refresh, logout, or a dedicated inquiry submission. This keeps the API consistent without forcing every action into CRUD semantics.

API namespaces

```
/api/v1/public/...   public read/submission endpoints
/api/v1/staff/...    protected management endpoints
/api/v1/auth/...     authentication endpoints
```

Initial public endpoints

```
GET  /api/v1/public/impact-metrics/
GET  /api/v1/public/opportunity-areas/
GET  /api/v1/public/stories/
GET  /api/v1/public/stories/<slug>/
GET  /api/v1/public/partners/
GET  /api/v1/public/events/
GET  /api/v1/public/events/<slug>/
POST /api/v1/public/inquiries/
POST /api/v1/public/newsletter/
```

Authentication endpoints

```
POST /api/v1/auth/login/
POST /api/v1/auth/refresh/
POST /api/v1/auth/logout/
```

Protected staff endpoints - initial shape

```
POST/PATCH/DELETE /api/v1/staff/stories/...
POST/PATCH/DELETE /api/v1/staff/events/...
POST/PATCH/DELETE /api/v1/staff/partners/...
POST/PATCH/DELETE /api/v1/staff/opportunity-areas/...
PATCH             /api/v1/staff/impact-metrics/<id>/
GET/PATCH         /api/v1/staff/inquiries/...
GET               /api/v1/staff/audit/...
```

Permission candidates

```
AllowAny
IsAuthenticated
IsStaffUser
IsContentManager
```

```
IsSuperAdmin
ReadOnlyOrStaff
```

JWT authenticates identity. Permissions authorize actions. Where relevant, object-level authorization must also be checked.

11. Authentication Architecture

Why both Django sessions and JWT

Django Admin is server-rendered by Django and should use Django's mature session/cookie authentication. The separate React staff portal is an independent API client and is the appropriate place for JWT access/refresh authentication.

Staff portal flow

```
React login form
    ↓
POST /api/v1/auth/login/
    ↓
Django validates credentials
    ↓
Access token + refresh strategy
    ↓
React requests protected resource
    ↓
Authorization: Bearer <access token>
    ↓
DRF authenticates user
    ↓
DRF permissions authorize action
```

Access and refresh

Use short-lived access credentials and a longer-lived refresh mechanism. Evaluate djangorestframework-simplejwt for implementation. Logout must be treated as a real server/client session decision; do not assume deleting a local string is sufficient if refresh credentials remain valid.

Browser token storage

Implementation decision still open

Do not blindly store long-lived sensitive tokens in localStorage. When JWT implementation begins, choose the browser storage/cookie strategy based on XSS risk, CSRF behavior, frontend/backend domains, HTTPS, refresh-token handling, and deployment architecture. Secure HttpOnly cookie handling for refresh credentials is a strong candidate, but the final design should be made deliberately.

Protected request decision flow

```
Request
  ↓
Valid authentication?
  ↓
Active account?
  ↓
Correct role/permission?
  ↓
Allowed to access/modify this object?
```

```
  ↓
Submitted fields allowed?
  ↓
Serializer validation passes?
  ↓
Perform action
  ↓
Write audit record
```

12. Security Philosophy

Security is a key identity of this project. It should be visible in the architecture, code, tests, and deployment decisions, not merely mentioned in the README.

Security principle

JWT itself does not make the site secure. Security comes from layered controls: server-side authorization, least privilege, safe credential handling, validation, secure configuration, throttling, upload restrictions, auditability, testing, patching, backups, and minimizing sensitive data.

Original 20-point security checklist considered

#	Security idea	TMK decision
1	Hide API keys	Required. Secrets stay out of frontend source and Git.
2	Purge Git secrets	Required. Prevent, scan, and rotate any accidentally exposed credential.
3	Use public DB key	Not applicable to current architecture. React never connects directly to PostgreSQL.
4	Enable row-level security	Conditional. Django/DRF permissions first; PostgreSQL RLS only if fine-grained/multi-tenant needs justify it.
5	Encrypt sensitive data	Required where appropriate; HTTPS in transit, provider/database protections, and avoid unnecessary sensitive data.
6	Enforce server-side auth	Critical. Backend is the authority.
7	Lock record access	Critical. Object-level permissions where required.
8	Block field tampering	Critical. Explicit serializer fields/read-only fields; never mass-assign sensitive properties.
9	Secure session cookies	Required for Django session workflows.
10	Hash passwords	Critical. Django password hashing; never plaintext/reversible storage.
11	Rate limit login	Required.
12	Add bot protection	Likely/conditional for public forms depending on spam/abuse.
13	Parameterize queries	Required; use Django ORM safely and avoid unsafe raw SQL.
14	Validate all input	Critical at backend; frontend validation is UX only.

15	Escape user content	Required. Avoid unsafe HTML injection and dangerous rendering.
16	Restrict file uploads	Critical.
17	Trim API responses	Required. Return only fields needed by the client.
18	Add security headers	Required in production.
19	Force HTTPS	Required in production.
20	Scan dependencies	Required as part of maintenance/GitHub workflow.

13. Additional Security Controls We Chose

Least privilege

Staff should receive only the permissions needed for their job. Working role ideas are SUPER_ADMIN, CONTENT_MANAGER, STAFF, and VIEWER, but keep the role model as simple as real responsibilities allow.

Audit logging

Record meaningful administrative changes and security-relevant actions. Example: who changed Youth Served from 15,487 to 15,621, when, and from what IP where appropriate. Centralize audit creation rather than scattering manual AuditLog.objects.create calls throughout views.

Safe error handling

Production clients should never see stack traces, secret values, database connection strings, filesystem paths, or detailed internal exceptions. Users receive safe messages; detailed diagnostics belong in server logs.

Backups and recovery

Security includes availability and recovery. Use managed PostgreSQL backups appropriate to the hosting provider and know how restoration works.

Dependency maintenance

Keep Django, DRF, React, and dependencies patched. Use GitHub/Dependabot or equivalent scanning where appropriate.

Security tests

Automated tests should explicitly cover unauthorized access, permission boundaries, expired/invalid authentication, field tampering, unpublished content exposure, invalid input, upload restrictions, and rate-limited endpoints.

Privacy/data minimization

TMK serves young people. Do not collect names, birthdays, addresses, schools, or other youth data merely because a form/database can store it. Collect and publish the minimum necessary information and confirm consent/usage rules for youth stories and photography.

MFA

Strong later-phase candidate for staff accounts after the core authentication flow is implemented correctly. Do not let MFA implementation block the initial public client delivery.

RLS

PostgreSQL Row-Level Security is not an MVP requirement. It becomes more compelling if TMK later has regional/departmental data boundaries or the application becomes multi-tenant.

14. File Upload Security

TMK will likely need story images, event images, and partner logos. File handling must be intentionally constrained.

- Allow only approved image formats such as JPEG, PNG, and WebP initially.
- Set maximum file size; exact limit should be chosen based on UX and hosting constraints (for example 5-10 MB, not yet final).
- Validate extension and actual content/MIME characteristics where practical.
- Generate/sanitize stored filenames rather than trusting user-supplied paths/names.
- Do not allow arbitrary executable uploads.
- Keep uploaded media separate from application code.
- Use local media only for development; production media should use durable object/cloud storage rather than ephemeral app-container disk.
- Optimize display sizes and use responsive/lazy-loaded images.
- Require authenticated/authorized staff for content uploads.
- If document uploads become a real feature later, design a separate policy and consider malware scanning.

15. Rate Limiting, Validation, CORS, HTTPS, and Headers

Rate limiting

Apply reasonable global API throttling plus tighter limits on abuse-prone endpoints. Do not choose arbitrary limits that block legitimate users; tune based on expected traffic and hosting-layer capabilities.

```
Higher-risk examples:
POST /api/v1/auth/login/
POST /api/v1/auth/refresh/
POST /api/v1/public/inquiries/
POST /api/v1/public/newsletter/
password reset endpoint if/when added
```

Validation layers

```
React validation
    ↓
User experience

DRF serializer validation
    ↓
Security + business/data correctness

Django model/database constraints
    ↓
Final integrity boundary
```

Never trust React validation. Attackers or testers can bypass React and call the API directly with Postman/curl.

CORS

Allow only known frontend origins in production. Development may allow localhost:5173. Do not ship permissive allow-all CORS without a requirement.

CSRF

Use Django CSRF protections where cookie/session-based workflows require them. JWT/cookie decisions must be evaluated together rather than treating CSRF as irrelevant.

HTTPS

All production traffic must use TLS/HTTPS.

Security headers

Configure appropriate production headers such as HSTS, content-type sniffing protections, frame protections, and a deliberate Content Security Policy strategy where feasible. Verify configuration against the actual deployment.

16. React Frontend Architecture

Use one React/Vite application with clearly separated public and staff experiences. This keeps deployment/repository management simpler while maintaining architectural boundaries.

Proposed folder structure

```
frontend/
├── public/
├── src/
│   ├── api/
│   │   ├── apiClient.js
│   │   ├── authApi.js
│   │   ├── impactApi.js
│   │   ├── storiesApi.js
│   │   ├── opportunitiesApi.js
│   │   ├── partnersApi.js
│   │   ├── eventsApi.js
│   │   └── inquiriesApi.js
│   ├── components/
│   │   ├── common/
│   │   ├── public/
│   │   └── staff/
│   ├── layouts/
│   │   ├── PublicLayout.jsx
│   │   └── StaffLayout.jsx
│   ├── pages/
│   │   ├── public/
│   │   └── staff/
│   ├── auth/
│   │   ├── AuthProvider.jsx
│   │   ├── ProtectedRoute.jsx
│   │   └── permissions.js
│   ├── hooks/
│   ├── utils/
│   ├── constants/
│   ├── styles/
│   ├── App.jsx
│   └── main.jsx
├── .env
├── .env.example
├── package.json
└── vite.config.js
```

API service rule

Do not scatter hard-coded fetch URLs throughout components. Components call domain API modules, which use one centralized API client. This makes base URL changes, auth headers, refresh behavior, and centralized error handling manageable.

HTTP client

Axios is the working choice because interceptors and centralized configuration are useful for JWT attachment/refresh and API error handling. Native fetch would also work, but Axios is selected for convenience and clarity in this architecture.

Authentication state

```
AuthProvider
- user
- isAuthenticated
- isLoading
- login()
- logout()
- refreshSession()
```

Protected routes

Unauthenticated staff navigation should redirect to /staff/login. Role-aware UI can hide unavailable actions, but Django still enforces every permission server-side.

State management

Start with local component state plus Context for authentication. Do not add Redux until the application has a genuine state-management problem.

17. React Components and UX Requirements

Public reusable components

```
Navbar
Footer
Button
PageHero
SectionHeading
ImpactStat
OpportunityCard
StoryCard
PartnerLogo
EventCard
CTASection
FormField
LoadingSpinner / Skeleton
ErrorMessage
EmptyState
```

Staff reusable components

```
Sidebar
Topbar
DashboardCard
DataTable
StatusBadge
FormField
FileUpload
ConfirmationModal
Pagination
SearchInput
FilterDropdown
Toast
AuditEntry
```

Required API-driven states

- Loading
- Success
- Empty
- Error
- Unauthorized/expired session
- Permission denied

A blank screen while data loads is not acceptable. Public-facing errors should be human-friendly; technical details stay in development logs/server diagnostics.

Responsive design

Build and test mobile, tablet, laptop, desktop, and large desktop. Public navigation should collapse appropriately on mobile. Staff navigation can use a collapsible sidebar.

Accessibility

- Semantic HTML

- Keyboard navigation
- Visible focus states
- Proper form labels
- Alt text for meaningful images
- Color contrast
- Correct button/link semantics
- Logical heading hierarchy
- Accessible validation/error messages

SEO

The public nonprofit site needs meaningful titles, meta descriptions, semantic headings, clean URLs, social/Open Graph metadata, sitemap/robots configuration, and performant pages. React + Vite remains the plan; evaluate prerendering/static-generation support later if SEO requirements demand it. Do not switch frameworks without a reason.

18. Public vs Staff Visual Language

Public site	Staff portal
Editorial and emotional	Functional and operational
Large photography	Tables, forms, status indicators
Whitespace and large typography	Efficient information density
Minimal calls to action	Clear CRUD/workflow actions
Institutional/premium	Professional business dashboard

They should share TMK's brand identity but should not look identical. The public site sells trust and mission; the staff portal supports work.

19. MVP and Delivery Phases

MVP - core client delivery

- Responsive public React site.
- Home, About, Impact, How We Build Opportunity, Stories, Partners, Events, Get Involved, Contact.
- Django REST API.
- PostgreSQL.
- Custom User model created before first migration.
- Django Admin.
- Impact management.
- Story management.
- Partner management.
- Event management.
- Inquiry storage.
- Secure public forms.
- Image uploads with restrictions.
- Baseline security controls.
- Backend tests including authorization/security cases.
- Postman API testing.
- Production deployment.

Phase 2 - custom staff experience

- Custom React staff portal.
- JWT login/refresh/logout flow.
- Protected CRUD dashboard.
- Audit log viewer.
- Role-aware staff UI.
- Expanded staff permissions.
- Improved impact visualization.
- Newsletter integration.

Phase 3 - only if client requirements justify

- MFA.
- Donation/payment integration.
- Scheduled publishing.
- Reusable media library.
- Advanced analytics.
- Notifications.
- Advanced bot protection.
- Fine-grained security monitoring.
- PostgreSQL RLS or other additional DB-level controls.

- Malware scanning if broader document uploads are introduced.

Features deliberately not forced into MVP

Do not let payment processing, complex CMS features, multiple sophisticated staff roles, media libraries, or advanced security tooling delay the actual public website unless the client says they are launch requirements.

20. Client Questions Still Unresolved

- What is the final TMK logo and approved brand package?
- What exact brand colors/hex values are approved?
- Which fonts are approved/licensed?
- What photography can TMK provide?
- Are photography and youth consent/usage releases documented?
- Which impact metrics and numbers are current and verified?
- What are the final official mission and vision statements?
- Which partners should appear publicly?
- What social accounts should be linked?
- What email address should receive contact/inquiry notifications?
- Does TMK already use a newsletter platform?
- Does TMK already have a donation/payment provider that should be linked rather than rebuilt?
- Which events should be seeded initially?
- Who should receive staff accounts?
- Who may publish, edit, delete, or manage staff?
- What youth-related personal information, if any, is truly necessary to collect or publish?
- What is the production domain/subdomain plan?
- What is the client's expected ongoing content-management workflow after launch?

Important data warning

The guideline includes alternate example impact figures and alternate copy/navigation wording. Those examples are concept material, not authoritative production data. Never silently choose one set as final. Build the system so TMK can manage verified values.

21. Testing Strategy

Backend functional tests

- Model constraints and validation.
- Serializer validation.
- Public list/detail endpoints.
- Publishing filters (unpublished content must not leak publicly).
- Staff CRUD behavior.
- Inquiry submission.
- Pagination where used.

Security/authorization tests

```
Anonymous PATCH impact metric          -> rejected
Authenticated non-staff PATCH metric -> rejected
Authorized staff PATCH metric        -> success
Anonymous GET published story        -> success
Anonymous GET unpublished story      -> not exposed
Forbidden field submitted manually   -> ignored/rejected
Invalid/oversized upload              -> rejected
Expired/invalid token                 -> rejected
Unauthorized audit modification       -> rejected
```

Postman

Use Postman throughout backend development to test public APIs, authentication, protected endpoints, validation errors, permission boundaries, token expiry/refresh, and tampered requests. Do not rely only on the React UI to prove the API is secure.

Frontend testing

Verify public navigation, responsive layouts, forms, loading/error/empty states, protected staff routing, permission-aware controls, and integration with the API. Add automated frontend tests for important behavior as the UI stabilizes.

22. Git, Secrets, and Repository Discipline

Git principles

- Initialize Git at the project root.
- Use a professional root .gitignore.
- Never commit .env, secrets, private keys, database credentials, production media credentials, or token secrets.
- Commit .env.example with names/placeholders only.
- Use small meaningful commits at logical milestones.
- Use feature branches when useful, but do not create process overhead for a solo first-client build.
- Keep README current with local setup, architecture, testing, and deployment instructions.

Secret exposure response

If a real credential is ever committed, removing it from the current file is not enough. Rotate/revoke the credential, remove it from reachable history where appropriate, and use secret scanning. Treat exposed secrets as compromised.

Planned first commit theme

After root structure, backend/frontend scaffolding, environment examples, and README are valid, use a clear commit such as: chore: initialize TMK full-stack project structure. Exact commit timing will be guided step-by-step during implementation.

23. Deployment and Production Principles

- Frontend likely Vercel or equivalent.
- Django API likely Render, Railway, or equivalent managed platform.
- Managed PostgreSQL.
- Production media on durable object/cloud storage rather than ephemeral application filesystem.
- Production DEBUG=False.
- Explicit ALLOWED_HOSTS and CORS origins.
- HTTPS enforced.
- Secure cookie/security-header configuration.
- Environment-based secrets.
- Database backups/recovery.
- Production-safe logging/error behavior.
- Dependency patching and monitoring.

Final providers are not sacred. Choose based on client budget, reliability, deployment simplicity, media/storage needs, and domain configuration.

24. Portfolio and Interview Intent

This project is the developer's first real client full-stack website and should become a flagship portfolio piece. The public result must be excellent for TMK first; the engineering should simultaneously demonstrate job-ready backend/full-stack skills.

Technologies the project should genuinely demonstrate

Python, Django, Django REST Framework, React, PostgreSQL, REST APIs, authentication, authorization, JWT, Django sessions, Git/GitHub, Postman, testing, secure configuration, deployment, audit logging, file handling, and responsive frontend architecture.

Target interview explanation

A strong future explanation is that the system separates a public React experience, a JWT-authenticated React staff SPA, and Django Admin as a reliable internal fallback. Public data is read through versioned REST endpoints, administrative writes are protected by server-side DRF permissions, PostgreSQL stores structured business data, and security was designed into authorization, validation, uploads, throttling, secrets, HTTPS, audit logs, and tests.

Do not resume-stuff

JWT exists because the staff SPA needs API authentication, not merely because it is a resume keyword. Django sessions remain for Django Admin because that is the correct tool. RLS, MFA, microservices, and other technologies are added only if requirements justify them.

25. Implementation Roadmap

9. Create the root project directory.
10. Initialize Git.
11. Create the GitHub repository.
12. Create the backend directory.
13. Create a Python virtual environment.
14. Install Django, DRF, PostgreSQL driver, environment configuration, and required development dependencies.
15. Create the Django project with split settings structure.
16. Create the domain apps: accounts, content, impact, events, engagement, audit.
17. Create the custom User model BEFORE the first migration.
18. Install/configure PostgreSQL locally and create the development database.
19. Configure environment variables and .env.example.
20. Run initial migrations and verify Django locally.
21. Register initial models/admin as they are implemented.
22. Create the React/Vite frontend.
23. Install React Router and selected frontend dependencies.
24. Create PublicLayout, route skeletons, and shared design foundation.
25. Create the centralized API client/service structure.
26. Connect React to the first public API endpoint.
27. Implement core models and public endpoints incrementally.
28. Build public pages incrementally using real API data.
29. Implement forms, validation, uploads, throttling, audit logging, and authorization tests.
30. Deploy MVP.
31. Build/finish the custom React staff portal and JWT flow as Phase 2.
32. Harden, document, test, and prepare portfolio/interview material.

26. Exact Next Step

Resume here

Begin Phase 1: Project Initialization. Do not restart planning unless a new requirement appears. Start with creating the root project directory and initializing Git/GitHub, then proceed through backend environment setup. Explain every command clearly and do not assume any files exist.

What has NOT happened yet

- No final root folder has been created as part of this handoff.
- No Git repository has been initialized for this implementation in this handoff.
- No Django project/apps have been created.
- No PostgreSQL development database has been configured.
- No migrations have been run.
- No React/Vite application has been created.
- No production credentials or client secrets should be assumed.
- No final brand assets, verified impact figures, or client-approved content should be assumed.

Recommended continuation prompt

```
Continue The Mills Korner full-stack project using this handoff as the source of truth.
Preserve the architecture and security decisions unless a new client requirement justifies a
change.
We are ready for Phase 1: Project Initialization.
Start with the root project directory and Git/GitHub setup, then guide me step-by-step.
Give exact commands and explain what each step does and why.
```

27. Decision Log

Decision	Status	Reason
React + Vite public frontend	Chosen	Experience, reusable UI, portfolio value.
Django + DRF backend	Chosen	Best fit for content/admin/security/relational client needs.
PostgreSQL	Chosen	Professional relational persistence.
REST /api/v1/	Chosen	Clear versioned contract.
Custom Django User from day one	Chosen	Avoid painful later auth-model migration.
Django Admin fallback	Chosen	Reliable early client management interface.
Custom React staff portal	Chosen, Phase 2	Professional client UX and full-stack experience.
JWT for staff SPA	Chosen	Legitimate API auth use case.
Django sessions for /admin/	Chosen	Correct server-rendered Django admin auth.
One React app for public + staff	Chosen	Simpler deployment while retaining route/layout separation.
Axios central API client	Working choice	Interceptors/configuration useful for auth/API handling.
Redux	Not initially	No demonstrated need.
Program model	Postponed	OpportunityArea sufficient until client identifies distinct programs.
MediaAsset model	Postponed	Direct image fields sufficient initially.
PostgreSQL RLS	Conditional	Use only if data boundaries justify DB-level enforcement.
MFA	Later candidate	Valuable, but not allowed to block secure core delivery.
Payment processing	Conditional/later	Prefer existing provider/link unless client requires custom integration.
Kubernetes/microservices/etc.	Rejected for now	Unnecessary complexity.

28. Source vs Engineering Distinction

Client-derived direction

- One organizing idea centered on opportunity.
- Visitor emotional journey from understanding to trust to participation.
- Elegant/quiet/confident visual direction.
- Large authentic photography, large typography, whitespace.
- Impact immediately after hero.
- Opportunity pathways rather than a cluttered program catalog.
- Transformation stories rather than generic testimonials.
- Partner credibility.
- Investment/support framing.
- Public live impact concept.
- Working brand palette direction and restrained typography.

Engineering decisions made during planning

- React + Vite.
- Django + DRF.
- PostgreSQL.
- Versioned REST API.
- Custom User model.
- Django Admin fallback.
- Custom React staff portal.
- JWT for staff SPA and sessions for Django Admin.
- Proposed database schema and Django app boundaries.
- Centralized API client, auth provider, protected routes.
- Security control set, audit logging, upload policy, throttling, testing strategy.
- MVP/Phase 2/Phase 3 boundaries.

Not final until client confirms

- Exact impact figures.
- Exact hero/supporting copy.
- Final navigation wording where guideline variants conflict.
- Final colors/hex values and fonts.
- Photography/assets and usage rights.
- Partner/event seed data.
- Staff roster and permission assignments.
- Donation/newsletter integrations.

29. Handoff Checklist for Any New Assistant

- Read this entire document before changing architecture.
- Treat client examples as examples unless marked verified.
- Keep security as a first-class concern.
- Do not trust frontend authorization.
- Create custom User before first migration.
- Keep Django Admin usable before custom staff portal.
- Do not expose unnecessary serializer fields.
- Do not store secrets in Git or React source.
- Do not connect React directly to PostgreSQL.
- Do not overengineer the MVP.
- Explain commands/files/code clearly and sequentially.
- Recommend Git commits at logical milestones.
- Use Postman plus automated tests.
- Keep the project production- and portfolio-quality.
- Continue at Phase 1 Project Initialization unless the user supplies a new requirement.